

1. The Shared-Worktree Problem

Parallel agents that edit the same checkout can overwrite, observe, or invalidate one another's intermediate state. Git worktrees provide separate working directories backed by the same repository metadata, allowing different branches or tasks to proceed without sharing the same files.

This solves file-system interference. It does not solve conceptual interference.

2. What Worktrees Guarantee

A worktree can isolate uncommitted changes and branch state. Agent A can modify the frontend while Agent B changes an independent parser without each seeing the other's half-finished files. Each can run tests against its own checkout.

The shared repository history still matters, and the eventual integration must reconcile both branches. A worktree is an isolation mechanism, not an automatic merge proof.

3. Semantic Conflict

Two branches can merge cleanly while producing a broken system. Agent A may rename a conceptual field in one layer; Agent B may add a feature based on the old semantics in another. Git sees different lines. The application sees a contradiction.

Therefore integration verification must be run on the combined state. "Both branches passed tests separately" does not imply "the merged result passes."

4. Choosing Task Boundaries

The cleanest parallel worktrees correspond to architectural boundaries with stable interfaces: documentation vs implementation, separate packages, independent test additions, or exploratory alternatives. The worst boundaries cut through a single invariant.

Ask whether each branch can define its contract with the rest of the system before work begins. If not, use sequential work or establish the interface first.

5. Transfer and Ignored Files

Codex worktree workflows can move or transfer work between the local checkout and a worktree. Ignored files deserve special attention because they may not follow ordinary Git movement. Local generated state, untracked fixtures, credentials, or build outputs can make two worktrees behave differently even when tracked files match.

Verification should therefore identify environment assumptions, not only commits.

6. Merge Procedure

A restrained integration procedure:

1. Review each branch against its delegated task.
2. Confirm each branch's relevant checks on its own state.
3. Inspect overlapping interfaces and assumptions before merging.
4. Combine the branches.
5. Run checks that exercise the interaction between them.
6. Run the broader repository checks justified by the combined change.
7. Perform a read-only review of the integrated diff when consequence warrants it.

This procedure is deliberately asymmetric: the final combined state receives verification that neither branch could perform alone.

7. Parallelism and Reasoning Effort

Parallel work is sometimes a better use of compute than increasing one agent's reasoning effort. Two focused explorations can cover independent hypotheses. In other cases, one High-effort agent with a coherent model of the system beats two Medium agents with duplicated discovery costs. The correct unit of optimization is successful verified work, not agent count.

References: OpenAI, Git worktrees in Codex; OpenAI, Codex subagents; OpenAI, Codex.

8. Alternative Implementations

Worktrees are especially useful when comparison itself is the goal. Agent A can implement a minimal adapter; Agent B can implement a deeper migration. Because each branch starts from the same base, the parent can compare diff size, test results, performance, and compatibility without one approach contaminating the other. The experiment is only fair if both branches are evaluated against the same acceptance criteria.

9. The Hidden-State Matrix

Before assuming two worktrees are equivalent, compare tracked files, untracked fixtures, ignored files, environment variables, service state, database state, dependency caches, and external credentials. A passing result in one worktree can be caused by local state rather than code. When a discrepancy appears, reduce it to the smallest state difference that predicts the result.

10. Merge Order

Merge order can matter even when commits do not conflict textually. If branch B was designed against behavior before branch A changed it, B should be revalidated after A lands. Treat integration as a new system state. The parent should not cite branch-local tests as final evidence after the state has changed.

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 12: Parallel Work

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>